



Concurrent Programming

(CT074-3-2-CCP)

Individual Assignment

Student Name : Lim Beng Rhui

TP Number : TP068495

Intake Code : APD2F2411CS(DA)

Lecturer Name : Dr. Kuan Yik Junn

Hand out Date : 6 May 2025

Hand in Date : 2 August 2025

Table of Contents

Introduction	3
Assumptions	4
Requirements Met	6
Additional Feature: Order Retry	6
Additional Feature: Stress Level Selection	6
Program Execution with Sample Output	7
Concurrency Concepts	13
Thread Management	13
Synchronisation Primitives	16
Resource Handling	21
Safety Aspects in Program Execution	24
Producer-Consumer Model	24
Race Condition Avoidance	25
Thread Scheduling Fairness	26
Thread Shutdown Protocol	27
Conclusion	28
References	29
Appendix A. Program Specification	31
Appendix B. Output Highlight	32

Introduction

With the rapid technological advancement, many businesses are shifting from physical stores to online platforms, leading to the rise of e-commerce platforms in recent years. Such a scenario has led to the increasing demand for dispatching delivery orders quickly, accurately, and efficiently, leading to the establishment of SwiftCart. The company is an e-commerce hub that is envisioned to specialise in processing and delivering packages via highly automated systems, where the company utilises different technologies like robotic arms and automated guided vehicles (AGV) to pack, label, sort and transport orders. Hence, this report details a comprehensive simulation of SwiftCart's warehouse system using the Java programming language, where multiple concurrency concepts were implemented to model real-time simultaneous operations across different components. By replicating real-world warehouse behaviour, the simulation aims to identify and address any constraints encountered within the warehouse system, so that SwiftCart can operate smoothly and efficiently when deploying its operations in a real-world context.

Assumptions

To define the scope and operational boundaries of the system, the following assumptions were made throughout the development of the e-commerce hub system:

- **Inventory Initialisation**

- The system initialises 16 unique items at the beginning, each with a fixed quantity.
- All items are stored on a shelf accessible by all picking arms.

- **System Resources**

- The system operates with four robotic picking arms, four order bins and three automated guided vehicles (AGV).
- All resources are finite and shared between threads.

- **Concurrency Model**

- Operations across all stages of the program follow a first-come, first-served approach to maintain fairness.

- **Order Generation**

- Six hundred delivery orders are generated at a constant rate of 2 orders per second.
- Each order includes details like the item list, payment method and shipping address.

- **Picking Operations**

- Each picking arm handles one order at a time.
- Items are picked by the picking arms from the shelf and placed into an available order bin.
- Each order bin can only contain items from one complete order at a time.
- If no order bins are available, the picking process halts until an order bin is freed.

- **Packing and Labelling**

- When an order bin is filled, the packing arm transfers its contents into a new unlabelled shipping box.
- All unlabelled shipping boxes will be scanned to ensure their contents match the original order.
- After the contents of a shipping box are checked, a label with destination details and a tracking number is assigned to the box.

- **Batching and Containerisation**

- Shipping boxes are divided into batches of 6 based on five regional zones: East Malaysia, Northern, Central, Southern and Eastern regions.
- Boxes are transferred from their batch into a container.
- Each container can hold up to 30 shipping boxes.
- The container is sealed when it is full or no new boxes are created.
- Once sealed, an automated guided vehicle (AGV) transfers the container to the loading bay.

- **Loading Bay and Trucks**

- Two loading bays are available, each having a capacity for one truck and five containers.
- When both loading bays are full, all packing operations are temporarily halted until a bay becomes available.
- Trucks are generated continuously unless blocked by the lack of space in the loading bays.
- Each truck can carry up to 18 containers before dispatching to other delivery hubs.

- **Failure and Recovery**

- There is a 1% probability for operational errors to occur, which may include:
 - Missing address or payment information during order creation.
 - Failure in picking items from the shelf.
 - Missing and malformed shopping box labels.
- Orders rejected during the order generation phase (due to missing order details) are given a second chance to be reprocessed via a retry mechanism. Orders that are rejected in subsequent phases are discarded directly.
- There is a 5% probability that an AGV will break down. Each breakdown requires a fixed repair time of 3 seconds to simulate downtime.

- **Additional Logging and Timing Control**

- No graphical user interface (GUI) is implemented. All operations and events are logged via console outputs and system logs.
- Each primary process introduces a fixed delay to simulate the time taken for real-world physical operations.

Requirements Met

The simulation of SwiftCart's warehouse system covers all the basic and additional requirements, as detailed in the program specification in Appendix A and the output highlights in Appendix B.

Additional Feature: Order Retry

If orders are rejected during the order creation phase, they are provided with another chance to be regenerated again. The code snippet for the `RetryOrdering` class is shown below.

```
public class RetryOrdering implements Runnable {
    @Override
    public void run() {
        Random randomObject = new Random();
        try {
            while (true) {
                Order rejectedOrder = DataManager.firstRejectedOrderList.take();
                if (rejectedOrder.equals(LAST_REJECTED_ORDER)) break;
                Logger.generateLog(REJECT_ORDER, rejectedOrder);

                if (randomObject.nextDouble() >= WRONG_DETAIL_PROBABILITY) {
                    String orderId = rejectedOrder.getOrderId();
                    rejectedOrder = Initializer.createRandomOrder(orderId);
                }

                rejectedOrder.markNotFirstTry();
                DataManager.orderQueue.put(rejectedOrder);
                Logger.generateLog(CREATE_ORDER, rejectedOrder);
                firstRejectCounter.incrementAndGet();
            }
        } catch (InterruptedException _) {
            // Code to handle exception here
        }
    }
}
```

Additional Feature: Stress Level Selection

Users are also given an additional option to select the stress level before the program executes, as shown in the output below. Based on the input, the program will declare the size of the orders and populate them accordingly.

```
Welcome to SwiftCart Warehouse Management System.

Before we start, let's select how you would like to populate the system.

-----
                        Stress Level
-----
1. Low stress level: 100 orders
2. Normal stress level: 600 orders
3. High stress level: 1000 orders

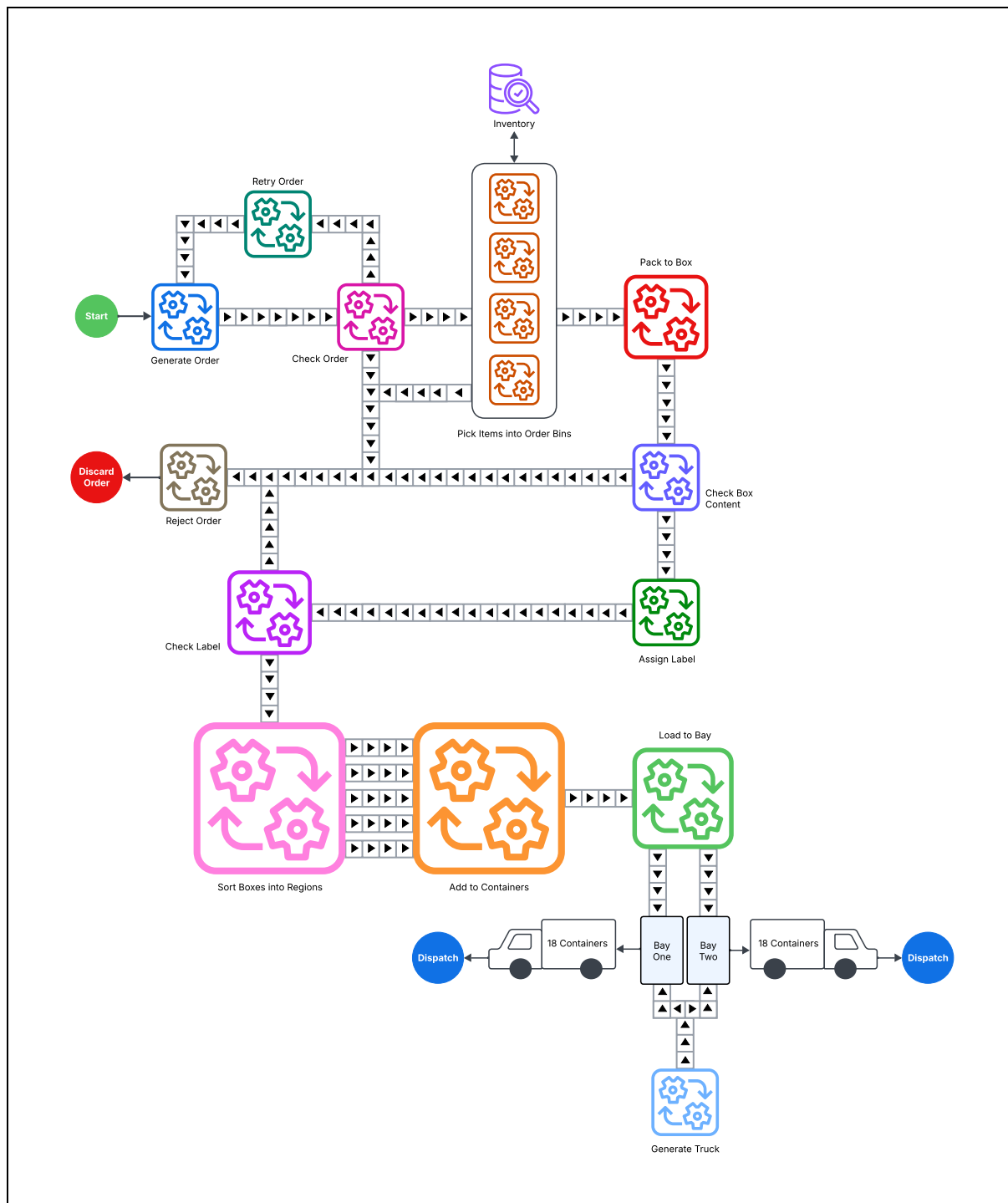
Please select the level:
```

Program Execution with Sample Output

The overall flow of the program is represented in the architectural diagram, as shown in Figure 1. The architectural diagram provides a high-level perspective of the system, enabling a more systematic approach for system development.

Figure 1

Architectural Diagram for SwiftCart's Warehouse System



The output representing each phase of the warehouse system is also captured and detailed, as follows.

```
Welcome to SwiftCart Warehouse Management System.

Before we start, let's select how you would like to populate the system.

-----
                        Stress Level
-----
1. Low stress level: 100 orders
2. Normal stress level: 600 orders
3. High stress level: 1000 orders

Please select the level: 2
```

When the program is executed, the system will prompt users to select a stress level for the system to populate the number of orders. To align with the requirements, level `2` is selected. The program then executes, where orders and trucks begin to populate. The output below shows the operations for the first order with `ORD0001` as its ID.

```
[Truck-Generation-Thread] Truck (TRUCK001) is created.
[Truck-Parking-Thread] Truck (TRUCK001) is parked at Bay (Bay 1).
[Order-Generation-Thread] Order (ORD0001) is created successfully.
[Picking-Arm-1] Picking Gaming Keyboard with quantity of 1 from the shelf for order (ORD0001).
[Picking-Arm-1] Picking Scented Candle with quantity of 1 from the shelf for order (ORD0001).
[Picking-Arm-1] Picking Fitness Tracker with quantity of 3 from the shelf for order (ORD0001).
[Picking-Arm-1] Picking Phone Charger with quantity of 2 from the shelf for order (ORD0001).
[Picking-Arm-1] Items from order ORD0001 placed into bin BIN001.
[Packing-Arm-Thread] Shipping box of order (ORD0001) is created.
[Packing-Scanner-Thread] Contents from the shipping box of order (ORD0001) is checked.
[Truck-Generation-Thread] Truck (TRUCK002) is created.
[Picking-Arm-2] Picking Phone Charger with quantity of 3 from the shelf for order (ORD0002).
[Truck-Parking-Thread] Truck (TRUCK002) is parked at Bay (Bay 2).
[Order-Generation-Thread] Order (ORD0002) is created successfully.
[Labelling-Thread] Shipping box of order (ORD0001) is labelled.
[Label-Check-Thread] Label of shipping box of order (ORD0001) is valid and checked.
[Picking-Arm-2] Items from order ORD0002 placed into bin BIN002.
[Batch-Distribute-Thread] Shipping box (TRACK0001) disposed to batch CENTRAL_REGION.
[Packing-Arm-Thread] Shipping box of order (ORD0002) is created.
[Packing-Scanner-Thread] Contents from the shipping box of order (ORD0002) is checked.
[Order-Generation-Thread] Order (ORD0003) is created successfully.
[Picking-Arm-3] Picking T-shirt with quantity of 2 from the shelf for order (ORD0003).
[Picking-Arm-3] Picking Portable Power Bank with quantity of 3 from the shelf for order (ORD0003).
[Truck-Generation-Thread] Truck (TRUCK003) is created.
[Truck-Parking-Thread] No available bay for parking for truck (TRUCK003).
```

Once order `ORD0001` is created successfully, the thread for picking arms, `Picking-Arm-1`, begins to pick up the items from the inventory and places them into an empty order bin, `BIN001`. Then, the packing arm thread, `Packing-Arm-Thread`, creates a shipping box for the order before passing it to the threads for labelling, checking, and dispatching to the `CENTRAL_REGION` batch. In the meantime, trucks, including `TRUCK001`, `TRUCK002` and `TRUCK003`, have also been generated. The truck with ID `TRUCK001` and `TRUCK002` parks successfully at `Bay 1` and `Bay 2` as the bays are not occupied yet. However, for `TRUCK003`, since both bays are already occupied by `TRUCK001` and `TRUCK002`, the truck `TRUCK003` will have to wait until one of the bays is available.

After dispatching to the batch, the operation for **ORD0001** temporarily halts, as the batch has to be filled with a sufficient number of boxes before it can be sent to containers and loaded into the respective truck. Hence, the system continues to populate the second order with ID **ORD0002**, and similar operations take place until the **CENTRAL_REGION** batch is full.

```
[Order-Generation-Thread] ] Order (ORD0002) is created successfully.
[Labelling-Thread] ] Shipping box of order (ORD0001) is labelled.
[Label-Check-Thread] ] Label of shipping box of order (ORD0001) is valid and checked.
[Picking-Arm-2] ] Items from order ORD0002 placed into bin BIN002.
[Batch-Distribute-Thread] ] Shipping box (TRACK0001) disposed to batch CENTRAL_REGION.
[Packing-Arm-Thread] ] Shipping box of order (ORD0002) is created.
[Packing-Scanner-Thread] ] Contents from the shipping box of order (ORD0002) is checked.
[Order-Generation-Thread] ] Order (ORD0003) is created successfully.
[Picking-Arm-3] ] Picking T-shirt with quantity of 2 from the shelf for order (ORD0003).
[Picking-Arm-3] ] Picking Portable Power Bank with quantity of 3 from the shelf for order (ORD0003).
[Truck-Generation-Thread] ] Truck (TRUCK003) is created.
[Truck-Parking-Thread] ] No available bay for parking for truck (TRUCK003).
[Labelling-Thread] ] Shipping box of order (ORD0002) is labelled.
[Label-Check-Thread] ] Label of shipping box of order (ORD0002) is valid and checked.
[Picking-Arm-3] ] Items from order ORD0003 placed into bin BIN003.
[Batch-Distribute-Thread] ] Shipping box (TRACK0002) disposed to batch CENTRAL_REGION.
[Packing-Arm-Thread] ] Shipping box of order (ORD0003) is created.
[Packing-Scanner-Thread] ] Contents from the shipping box of order (ORD0003) is checked.
[Order-Generation-Thread] ] Order (ORD0004) is created successfully.
[Picking-Arm-4] ] Picking Coffee Mug with quantity of 4 from the shelf for order (ORD0004).
[Picking-Arm-4] ] Picking Fitness Tracker with quantity of 3 from the shelf for order (ORD0004).
[Labelling-Thread] ] Shipping box of order (ORD0003) is labelled.
[Label-Check-Thread] ] Label of shipping box of order (ORD0003) is valid and checked.
[Picking-Arm-4] ] Items from order ORD0004 placed into bin BIN004.
[Batch-Distribute-Thread] ] Shipping box (TRACK0003) disposed to batch NORTHERN_REGION.
[Packing-Arm-Thread] ] Shipping box of order (ORD0004) is created.
[Packing-Scanner-Thread] ] Contents from the shipping box of order (ORD0004) is checked.
[Order-Generation-Thread] ] Order (ORD0005) is created successfully.
[Picking-Arm-1] ] Picking Water Bottle with quantity of 4 from the shelf for order (ORD0005).
[Labelling-Thread] ] Shipping box of order (ORD0004) is labelled.
[Label-Check-Thread] ] Label of shipping box of order (ORD0004) is valid and checked.
[Picking-Arm-1] ] Items from order ORD0005 placed into bin BIN001.
[Batch-Distribute-Thread] ] Shipping box (TRACK0004) disposed to batch EAST_MALAYSIA.
[Packing-Arm-Thread] ] Shipping box of order (ORD0005) is created.
[Packing-Scanner-Thread] ] Contents from the shipping box of order (ORD0005) is checked.
[Order-Generation-Thread] ] Order (ORD0006) is created successfully.
[Picking-Arm-2] ] Picking Bluetooth Headphones with quantity of 1 from the shelf for order (ORD0006).
[Picking-Arm-2] ] Picking Backpack with quantity of 3 from the shelf for order (ORD0006).
[Picking-Arm-2] ] Picking LED Desk Lamp with quantity of 2 from the shelf for order (ORD0006).
[Labelling-Thread] ] Shipping box of order (ORD0005) is labelled.
[Label-Check-Thread] ] Label of shipping box of order (ORD0005) is valid and checked.
[Picking-Arm-2] ] Items from order ORD0006 placed into bin BIN002.
[Batch-Distribute-Thread] ] Shipping box (TRACK0005) disposed to batch CENTRAL_REGION.
```

As shown in the output above, orders with ID **ORD0002** to **ORD0005** experienced the same process as **ORD0001**. Each picking operation is handled by different picking arms, which indicates that the thread scheduling mechanism for picking arms is fair.

```
[Batch-Distribute-Thread] ] Shipping box (TRACK0026) disposed to batch CENTRAL_REGION.
[Packing-Arm-Thread] ] Shipping box of order (ORD0027) is created.
[Order-Final-Rejected-Thread] ] Order (ORD0027) is rejected with reason: Packing-Scanner-Thread detects different match.
[Order-Generation-Thread] ] Order (ORD0028) is created successfully.
[Picking-Arm-4] ] Picking Smartphone Stand with quantity of 2 from the shelf for order (ORD0028).
[Picking-Arm-4] ] Picking Yoga Mat with quantity of 1 from the shelf for order (ORD0028).
```

Rejection operations are also simulated by the program, where if we take a look at order **ORD0027**, the order is rejected by the **Order-Final-Rejected-Thread** as the **Packing-Scanner-Thread** detects that the items in the order bin do not match the order.

Similar operations were performed towards all subsequent orders after `ORD0005` , including the 16th order with ID `ORD0016` . After the order `ORD0016` is checked, packed and dispatched, the `NORTHERN_REGION` batch is already full. Hence, the `Container-Generation-Thread` thread loads all the orders from the `NORTHERN_REGION` batch into container `C001` , as shown below.

```
[Labelling-Thread           ] Shipping box of order (ORD0016) is labelled.
[Label-Check-Thread        ] Label of shipping box of order (ORD0016) is valid and checked.
[Picking-Arm-1             ] Items from order ORD0017 placed into bin BIN001.
[Batch-Distribute-Thread   ] Shipping box (TRACK0016) disposed to batch NORTHERN_REGION.
[Container-Generation-Thread] Batch with order (TRACK0003) is loaded to Container C001.
[Container-Generation-Thread] Batch with order (TRACK0007) is loaded to Container C001.
[Container-Generation-Thread] Batch with order (TRACK0008) is loaded to Container C001.
[Container-Generation-Thread] Batch with order (TRACK0009) is loaded to Container C001.
[Container-Generation-Thread] Batch with order (TRACK0010) is loaded to Container C001.
[Container-Generation-Thread] Batch with order (TRACK0016) is loaded to Container C001.
```

Once the container `C001` is full, it is loaded into one of the trucks parked at the bay. Based on the output below, the container is loaded into `Bay 1` , before the `Truck-Loading-Thread` thread moves the container into the truck parked at `Bay 1` , which is `TRUCK001` .

```
[Label-Check-Thread        ] Label of shipping box of order (ORD0135) is valid and checked.
[Picking-Arm-4             ] Items from order ORD0136 placed into bin BIN004.
[Batch-Distribute-Thread   ] Shipping box (TRACK0132) disposed to batch NORTHERN_REGION.
[Container-Generation-Thread] Batch with order (TRACK0110) is loaded to Container C006.
[Container-Generation-Thread] Batch with order (TRACK0116) is loaded to Container C006.
[Container-Generation-Thread] Batch with order (TRACK0122) is loaded to Container C006.
[Container-Generation-Thread] Batch with order (TRACK0130) is loaded to Container C006.
[Container-Generation-Thread] Batch with order (TRACK0131) is loaded to Container C006.
[Container-Generation-Thread] Batch with order (TRACK0132) is loaded to Container C006.
[AGV-1                     ] Container C001 is loaded to Bay (Bay 1).
[Truck-Loading-Thread      ] Container C001 is loaded to Truck (TRUCK001).
[Packing-Arm-Thread        ] Shipping box of order (ORD0136) is created.
[Packing-Scanner-Thread    ] Contents from the shipping box of order (ORD0136) is checked.
```

Lastly, trucks shall dispatch once the number of containers loaded reaches maximum capacity. However, since the program only simulates 600 orders, where each container contains 30 boxes, it is estimated that the number of containers that can be produced is:

$$600 \text{ orders} \div 30 \text{ orders per container} = 20 \text{ containers}$$

Since the containers are equally distributed into each bay, it is estimated that each bay will only have approximately 10 containers. This indicates that both trucks will probably be unable to load containers to their maximum capacity and will only be forced to depart when the program ends. The output below accurately depicts the aforementioned scenario, where `TRUCK001` and `TRUCK002` depart from the loading bays only when the program ends, indicating the trucks still have remaining capacities.

```

[Packing-Arm-Thread          ] Shipping box of order (ORD0600) is created.
[Packing-Scanner-Thread      ] Contents from the shipping box of order (ORD0600) is checked.
[Labelling-Thread            ] Shipping box of order (ORD0600) is labelled.
[Container-Generation-Thread ] Batch with order (TRACK0549) is loaded to Container C017.
[Container-Generation-Thread ] Batch with order (TRACK0571) is loaded to Container C017.
[Container-Generation-Thread ] Batch with order (TRACK0570) is loaded to Container C021.
[Container-Generation-Thread ] Batch with order (TRACK0572) is loaded to Container C021.
[Container-Generation-Thread ] Batch with order (TRACK0574) is loaded to Container C021.
[Container-Generation-Thread ] Batch with order (TRACK0568) is loaded to Container C018.
[Container-Generation-Thread ] Batch with order (TRACK0575) is loaded to Container C018.
[Container-Generation-Thread ] Batch with order (TRACK0576) is loaded to Container C018.
[Container-Generation-Thread ] Batch with order (TRACK0577) is loaded to Container C018.
[Container-Generation-Thread ] Batch with order (TRACK0573) is loaded to Container C020.
[Container-Generation-Thread ] Batch with order (TRACK0579) is loaded to Container C020.
[Container-Generation-Thread ] Batch with order (TRACK0580) is loaded to Container C020.
[Container-Generation-Thread ] Batch with order (TRACK0581) is loaded to Container C020.
[Container-Generation-Thread ] Batch with order (TRACK0537) is loaded to Container C019.
[Container-Generation-Thread ] Batch with order (TRACK0544) is loaded to Container C019.
[Container-Generation-Thread ] Batch with order (TRACK0557) is loaded to Container C019.
[Container-Generation-Thread ] Batch with order (TRACK0578) is loaded to Container C019.
[AGV-2                      ] Container C017 is loaded to Bay (Bay 1).
[AGV-1                      ] Container C019 is loaded to Bay (Bay 1).
[AGV-2                      ] Container C020 is loaded to Bay (Bay 2).
[Truck-Loading-Thread        ] Container C017 is loaded to Truck (TRUCK001).
[Truck-Loading-Thread        ] Container C018 is loaded to Truck (TRUCK002).
[Truck-Loading-Thread        ] Container C019 is loaded to Truck (TRUCK001).
[Truck-Loading-Thread        ] Container C020 is loaded to Truck (TRUCK002).
[Truck-Loading-Thread        ] Container C021 is loaded to Truck (TRUCK001).
[AGV-3                      ] Container C018 is loaded to Bay (Bay 2).
[AGV-1                      ] Container C021 is loaded to Bay (Bay 1).
[Truck-Parking-Thread        ] Truck (TRUCK003) departed without parking.
[Truck-Dispatch-Thread       ] Truck (TRUCK001) departed.
[Truck-Dispatch-Thread       ] Truck (TRUCK002) departed.

```

```

~~~~~
Picking Arms executor service shutdown completed.
AGV Loaders executor service shutdown completed.
All threads have been terminated successfully!
~~~~~

```

The final statistics will be displayed at the program's end, as shown in the output below. The statistics first show that all the executor services and threads are terminated successfully, indicating that all the tasks have been completed and the threads did not run into any deadlocks or infinite waiting. Then, the program shows the final output that includes several elements, including the processing summary that shows the total amount of items that are handled throughout the simulation, performance metrics for trucks, AGVs and the entire system to measure if any performance issues are occurring during program execution, and the system clearance verification that verifies if there exists any dangling objects that has not be appropriately handled by the system when all threads end.

```

Picking Arms executor service shutdown completed.
AGV Loaders executor service shutdown completed.
All threads have been terminated successfully!

=====
SWIFT CART E-COMMERCE CENTRE - FINAL STATISTICS REPORT
=====

-----
PROCESSING SUMMARY
-----
- Orders received & processed : 600
- Orders given second chance : 18
- Boxes sent to packing : 600
- Orders completed & shipped : 581
- Orders ultimately rejected : 19
- Containers shipped : 21
- Trucks completed delivery : 2

-----
TRUCK PERFORMANCE METRICS
-----
- Minimum loading time : 125 ns
- Maximum loading time : 166 ns
- Average loading time : 145.50 ns
- Minimum wait time : 0 ms
- Maximum wait time : 301698 ms
- Average wait time : 100566.33 ms

-----
AGV PERFORMANCE
-----
- Total AGV breakdowns : 3
- Total downtime : 7239 ms

-----
SYSTEM PERFORMANCE
-----
- Total runtime : 305129 ms (305.13 seconds)
- Orders processed per second : 1.97

-----
SYSTEM CLEARANCE VERIFICATION
-----
- Orders in queue : 0
- Order bins in queue : 4
- Unchecked shipping boxes : 0
- Checked shipping boxes : 0
- Labelled shipping boxes : 0
- Checked label shipping boxes : 0
- Containers in overall queue : 0
- Containers in Bay 1 : 0
- Containers in Bay 2 : 0
- Trucks in queue : 0

=====
System cleared: YES
=====

Process finished with exit code 0

```

Concurrency Concepts

This section explores the concurrency concepts implemented in the project to support the safe and efficient simultaneous execution of multiple tasks.

Thread Management

Several mechanisms are implemented to control how threads are created and managed in the project, including:

- **Thread Class**

Threads represent the most fundamental unit of execution in a program (Microsoft, 2022). In this project, threads are implemented extensively to enable different operations to operate simultaneously, allowing the system to simulate real-world warehouse operations more reliably and efficiently.

In Java, threads can be created directly using the `Thread` class, where such an approach of creating threads is preferred when the program requires simple threads to perform multiple lightweight tasks simultaneously (DataCamp, 2025). Since most tasks only require one thread to be executed for each of them, this method is implemented extensively in the current project under the `ThreadManager` class. A new `Thread` object can be instantiated by passing a task, which can either be defined by extending the `Thread` class or by implementing the `Runnable` interface, to the `Thread` constructor. Then, an optional thread name can be provided to allow easy monitoring and debugging.

```
public class ThreadManager {  
    private static final Thread populateOrderThread = new Thread(  
        new OrderGeneration(), "Order-Generation-Thread"  
    );  
    public static void startAll() {  
        populateOrderThread.start();  
    }  
}
```

The code snippet above demonstrates how the `populateOrderThread` thread is initialised, where the `Thread` object, given the name of `Order-Generation-Thread`, will be executing the processes defined by the `run()` method in the `OrderGeneration` class. Once the `startAll()` function is executed, the `start()` method is called on the `populateOrderThread` thread to invoke the `Thread` object and begin its task.

- **Runnable Interface**

As one of the core components in Java concurrency, the **Runnable** interface provides a clean and modular manner for users to declare tasks that a thread can execute. An advantage of implementing the **Runnable** interface instead of extending the **Thread** class is that Java does not support multiple inheritance. In other words, if tasks are declared by extending the **Thread** class, the associated class cannot inherit from other superclass, limiting their functionality and reusability (Lahmidi, 2024).

```
public class PackingScanner implements Runnable {

    @Override
    public void run() {
        try {
            while (true) {

                ShippingBox retrievedBox = DataManager.uncheckedShippingBoxQueue.take();
                if (retrievedBox == LAST_SHIPPING_BOX) break;
                Map<Item, Integer> orderContent = retrievedBox.getOrder().getItemList();
                Map<Item, Integer> boxContent = retrievedBox.getItemList();

                if (orderContent != null && boxContent != null &&
                    orderContent.size() == boxContent.size() &&
                    orderContent.entrySet().stream().allMatch(
                        entry -> entry.getValue().equals(boxContent.get(entry.getKey())))) {

                    DataManager.checkedShippingBoxQueue.put(retrievedBox);
                    Logger.generateLog(CHECK_SHIPPING_BOX, retrievedBox);

                } else {

                    RejectedOrder rejectedOrder = new RejectedOrder(
                        retrievedBox.getOrder(),
                        Thread.currentThread().getName() + " detects different match.");
                    DataManager.secondRejectedOrderList.put(rejectedOrder);

                }

            }
        } catch (InterruptedException _) {
            // Code to handle exception
        } finally {
            try {
                DataManager.checkedShippingBoxQueue.put(LAST_SHIPPING_BOX);

            } catch (InterruptedException _) {
                // Code to handle exception
            }

        }

    }

}
```

In this project, all concurrent operations, including the **PackingScanner** task shown in the code snippet above, are implemented using the **Runnable** interface. This promotes a more modular design that allows the task logic to be encapsulated cleanly and flexible integration with other constructs like **ExecutorService** for more advanced thread handling and resource pooling.

- **ExecutorService with ThreadFactory**

The **ExecutorService** framework is provided by Java to assist programmers in simplifying thread management and improving program scalability, as the framework can automate the execution of tasks by automatically assigning them to different threads within a managed thread pool. When a similar task has to be carried out by multiple threads, programmers can submit **Runnable** tasks directly into an **ExecutorService** object that can handle thread lifecycle and resource usage automatically, instead of creating multiple threads and handling their executions manually (Tuchin, 2024). Such an approach reduces the risk of concurrency issues like thread starvation, which takes place easily when the threads in the system are not designed and managed appropriately.

```
public class ThreadManager {  
  
    public final static int NUMBER_OF_AGV = 3;  
    private static ExecutorService agvArms;  
  
    public static void startAll() {  
        agvArms = Executors.newFixedThreadPool(NUMBER_OF_AGV, new ThreadFactory() {  
            private int counter = 1;  
  
            @Override  
            public Thread newThread(Runnable r) {  
                Thread t = new Thread(r, "AGV-" + counter++);  
                t.setDaemon(false);  
                return t;  
            }  
        });  
  
        for (int i = 0; i < NUMBER_OF_AGV; i++) agvArms.submit(new TransferToBay());  
    }  
}
```

As demonstrated in the code snippet above, since the program requires the simulation of three automated guided vehicles (AGVs) to transfer containers to the loading bay, **ExecutorService** is implemented to simplify the process of handling the three AGV threads. The **ExecutorService** is first initialised with a fixed-size thread pool, with a customised **ThreadFactory** to assign unique names to each thread (e.g. **AGV-1** , **AGV-2** , etc.) while ensuring the threads are not daemon threads to avoid them from automatically halting when the main thread ends. Programmers can then easily create multiple threads for AGV by using the **submit()** method without requiring manual thread instantiation, improving the system's scalability without overloading the system with unnecessary manual thread creations and reducing the risk of thread management errors.

Synchronisation Primitives

Several synchronisation primitives are implemented in the project to provide controlled access to shared resources while ensuring thread interactions are coordinated.

- **synchronized Keyword**

The `synchronized` keyword is often implemented in concurrent programs to ensure mutual exclusion, where only one thread can execute a particular section of code at a time. Behind the scenes, when a thread tries to enter the block of code marked with the `synchronized` keyword, the thread must first acquire a lock on the associated object. If another thread already holds the lock, the current thread will have to wait until the lock is released before it can proceed and access the code (Coffey, 2021).

In this project, the `synchronized` keyword is widely used to guard code blocks involving shared resources. An example is the `Bay` class, which simulates a loading bay handling trucks and containers.

```
public class Bay {  
  
    private final String name;  
    private Truck currentTruck;  
    private final LinkedBlockingQueue<Container> containerQueue;  
  
    public synchronized Truck getCurrentTruck() {  
        return currentTruck;  
    }  
  
    public synchronized boolean isAvailableForTruck() {  
        return currentTruck == null;  
    }  
  
    public synchronized void parkTruck(Truck truck) {  
        currentTruck = truck;  
    }  
  
    public synchronized void truckDeparted() {  
        currentTruck = null;  
    }  
}
```

In the code snippet, methods like `parkTruck()` and `truckDeparted()` are marked as `synchronized` to avoid race conditions, where multiple threads attempt to perform concurrent modification towards the same `Bay` object instance. For example, suppose two threads attempt to park and remove a truck from the bay concurrently. In that case, the `synchronized` keyword ensures that only one of the threads can make modifications at a time to maintain the correctness and consistency of the system.

- **volatile Keyword**

The **volatile** keyword is another Java concurrency primitive used to ensure that any changes made towards a variable will be visible among all threads. According to Jenkov (2023), the **volatile** keyword ensures that when the content of a variable is changed, the program will immediately reflect the changes to the main memory, and the program must retrieve the content from the main memory when accessing the variable. This prevents the program from reading values cached inside the thread's local memory. While using the **volatile** keyword alone does not ensure mutual exclusion, it still acts as a crucial component when the program needs to update simple flags or the state of variables that will be accessed and modified by multiple threads.

```
public class TruckDispatch implements Runnable {
    public static volatile boolean stopFlag = false;
    private static final AtomicBoolean alternateFlag = new AtomicBoolean(true);

    @Override
    public void run() {
        try {
            while (!stopFlag) {
                // Code to handle truck dispatch
            }
        } finally {
            // Code to clean up remaining trucks at both bays
        }
    }

    public static void stopRunning() {
        stopFlag = true;
    }
}
```

The code snippet above demonstrates the use of the **volatile** keyword in the **TruckDispatch** class. In the system, when all orders are generated, the thread running the **TruckLoading** task, which is the **Truck-Loading-Thread**, will invoke the **stopRunning()** method to inform the thread running the **TruckDispatch** task, which is the **Truck-Dispatch-Thread**, to dispatch all the parked trucks and end the program. In this case, both **Truck-Dispatch-Thread** and **Truck-Loading-Thread** are running simultaneously. Without the **volatile** keyword, when the **Truck-Loading-Thread** thread changes the **stopFlag** variable to **true**, the **Truck-Dispatch-Thread** thread might not detect the change due to CPU caching, leading to the **Truck-Dispatch-Thread** thread to perform the checking infinitely without able to exit, which leads to inconsistent or unexpected behaviour and break the safeness of the entire program.

- **Atomic Variable Classes**

In a multi-threaded application, if the read-modify-write operations of variables are not handled atomically, race conditions may occur, where two or more threads attempt to modify the same data simultaneously, leading to inconsistent results. Aside from using the `synchronized` keyword, Java offers another construct that enables programmers to make modifications towards single variables safely: atomic variable classes like `AtomicBoolean`, `AtomicInteger` and `AtomicLong`. These classes utilise the compare-and-swap (CAS) method, where the program will check to determine if the variable's value is the expected value. The operation continues if both values match. Otherwise, no action will be performed towards the variable (Drosopoulou, 2024). This ensures that the updates performed on these variables are atomic without requiring additional explicit synchronisation to be implemented.

```
public class Container {  
  
    private static final AtomicInteger indexNum = new AtomicInteger(1);  
    private final ArrayBlockingQueue<ShippingBox> shippingBoxList;  
    private final String name;  
    private final Address.Region region;  
    private final AtomicBoolean loadedToBay = new AtomicBoolean(false);  
  
    public boolean markLoadedToBay() {  
        return loadedToBay.compareAndSet(false, true);  
    }  
  
    private String generateName() {  
        return String.format("C%03d", indexNum.getAndIncrement());  
    }  
}
```

Taking the `Container` class as an example, as shown in the code snippet above, the class implements two types of atomic variables. The `indexNum` is of the `AtomicInteger` type that has an initial value of `1`, which ensures that when different threads create a new `Container` object, the containers will have a unique container ID, as any concurrent access to the `indexNum` will be handled safely by the `AtomicInteger` class. In other words, only one thread can access and increment the `indexNum` variable at a time. Similarly, the `loadedToBay` variable, which is of `AtomicBoolean`, also prevents race conditions from occurring, as even though the `markedLoadedToBay()` method is accessed by multiple threads concurrently, the `compareAndSet()` method invoked will only perform operations on the `loadedToBay` variable if the value of the `loadedToBay` variable is `false`. Once the value of `loadedToBay` is switched to `true`, the `markLoadedToBay()` method will no longer modify `loadedToBay`.

- **ReentrantLock Lock**

In addition to the `synchronized` keyword, Java also provides the `ReentrantLock` class that offers a more flexible and powerful locking mechanism to implement mutual exclusion. Beyond the features offered by `synchronized`, `ReentrantLock` supports condition-based signalling via methods like `signalAll()` and `await()` to control thread execution. As shown in the code snippet below, when the truck queue is full, the lock is called on `await()` to wait until `signalAll()` is called for available space.

```
public class TruckGenerator implements Runnable {
    private static final Lock lock = new ReentrantLock();
    private static final Condition truckParkingCondition = lock.newCondition();
    private static volatile boolean availableSpace = true;
    private static volatile boolean isRunning = true;

    @Override
    public void run() {
        try {
            while (isRunning) {
                lock.lock();
                try {
                    while (!availableSpace && isRunning) truckParkingCondition.await();
                    // Code to handle truck generation
                } finally {
                    lock.unlock();
                }
                TimeUnit.MILLISECONDS.sleep(TRUCK_GENERATION_TIME_MS);
            }
        } catch (InterruptedException _) {
            // Code to handle exceptions
        } finally {
            // Code to handle terminations
        }
    }

    public static void continueGeneration() {
        lock.lock();
        try {
            availableSpace = true;
            truckParkingCondition.signalAll();
        } finally {
            lock.unlock();
        }
    }

    public static void stopGeneration() {
        lock.lock();
        try {
            availableSpace = false;
        } finally {
            lock.unlock();
        }
    }

    public static void notifyStop() {
        lock.lock();
        try {
            isRunning = false;
            truckParkingCondition.signalAll();
        } finally {
            lock.unlock();
        }
    }
}
```

- **Wait and Notify Mechanism**

In concurrency programming, there are situations where a thread has to wait for a particular condition before it can continue its execution. To handle such scenarios, Java has provided the `wait()`, `notify()` and `notifyAll()` methods, allowing threads to be paused and resumed based on shared resources or conditions.

```
public class PackingArms implements Runnable {

    private static final Object lock = new Object();
    private volatile static boolean isRunning = true;

    @Override
    public void run() {
        try {
            while (true) {
                synchronized (lock) {
                    while (!isRunning) {
                        Logger.generateLog(Logger.Type.PACKING_STOPPED);
                        lock.wait();
                    }
                    // Code to handle packing order operation
                }
            }
        } catch (InterruptedException _) {
            // Code to handle exceptions
        } finally {
            try {
                DataManager.uncheckedShippingBoxQueue.put(LAST_SHIPPING_BOX);
            } catch (InterruptedException _) {
                // Code to handle exceptions
            }
        }
    }

    public static void stopRunning() {
        synchronized (lock) {
            isRunning = false;
            lock.notifyAll();
        }
    }

    public static void startRunning() {
        synchronized (lock) {
            boolean previousStatus = isRunning;
            isRunning = true;
            lock.notifyAll();
            if (!previousStatus) Logger.generateLog(Logger.Type.PACKING_STARTED);
        }
    }
}
```

In the program, the packing operations must be paused when the loading bays are full. Hence, the `stopRunning()` method is declared to set the `isRunning` flag to `false` so that the thread successfully enters the waiting state. Once the loading bays have empty capacity, the `startRunning()` method is called, where the `isRunning` flag turns `true` and the `notifyAll()` method is called, allowing the thread to continue its execution.

Resource Handling

To effectively handle different objects in the program without leading to unexpected behaviours, several Java concurrency utilities, such as semaphores, blocking queues and concurrent hash maps, are implemented.

- **Semaphore**

A `Semaphore` object is a concurrency utility provided by Java that controls and allocates shared resources to a limited number of threads at a time. In simple words, a `Semaphore` object works like a gatekeeper, which plays a crucial role in concurrency as limiting the access towards resources not only ensures that threads can perform their tasks in an orderly manner, but also helps to prevent resource contention, where multiple threads over-occupy resources like CPU time and memory usage, potentially leading to performance degradation (CodeSignal, 2015).

```
public class DataManager {  
    public static final Semaphore pickingRoboticArmSemaphore = new Semaphore(PICKING_ARM_SIZE);  
}
```

```
public class PickingArms implements Runnable {  
  
    @Override  
    public void run() {  
        try {  
            while (true) {  
                Order order = orderQueue.take();  
                DataManager.pickingRoboticArmSemaphore.acquire();  
  
                try {  
                    // Code to handle picking arm operations  
                } catch (IllegalArgumentException _) {  
                    // Code to handle errors  
                } finally {  
                    DataManager.pickingRoboticArmSemaphore.release();  
                }  
            }  
        } catch (InterruptedException _) {  
        } finally {  
            // Code to handle ending operations  
        }  
    }  
}
```

As shown in the code snippet above, the `pickingRoboticArmSemaphore` semaphore is initialised with a fixed number of permits, which is four in this case, indicating that the semaphore can be acquired four times in total using the `acquire()` method. If there are more than four attempts, the `pickingRoboticArmSemaphore` semaphore will block the thread until one of the permits is released via the `release()` method, ensuring that the operations are controlled and consistent.

- **Blocking Queue**

A `BlockingQueue` is a thread-safe data structure provided by Java to support users in performing concurrency operations without manually synchronizing across different threads. `BlockingQueue` is different from regular queues because it delivers built-in support for automatic blocking, where if a thread attempts to remove an item from an empty queue, `BlockingQueue` will block the thread until a new element is inserted into the queue (Musib, 2024).

```
public class DataManager {  
    public static final BlockingQueue<ShippingBox> checkedShippingBoxQueue = new LinkedBlockingQueue<>();  
    public static final BlockingQueue<ShippingBox> labelledShippingBoxQueue = new LinkedBlockingQueue<>();  
}
```

```
public class SupplyBoxLabelling implements Runnable {  
    private static final Random randomObject = new Random();  
    @Override  
    public void run() {  
        try {  
            while (true) {  
                ShippingBox retrievedBox = DataManager.checkedShippingBoxQueue.take();  
                if (retrievedBox == LAST_SHIPPING_BOX) break;  
                TimeUnit.MILLISECONDS.sleep(LABELLING_STATION_DURATION_MS);  
                if (randomObject.nextDouble() >= WRONG_DETAIL_PROBABILITY)  
                    retrievedBox.setLabel();  
                Logger.generateLog(Logger.Type.LABEL_SHIPPING_BOX, retrievedBox);  
                DataManager.labelledShippingBoxQueue.put(retrievedBox);  
            }  
        } catch (InterruptedException _) {  
            // Code to handle exception  
        } finally {  
            try {  
                DataManager.labelledShippingBoxQueue.put(LAST_SHIPPING_BOX);  
            } catch (InterruptedException _) {  
                // Code to handle exception  
            }  
        }  
    }  
}
```

The code snippet above shows the `SupplyBoxLabelling` class, where shipping boxes will be labelled with their respective tracking IDs and destinations before proceeding to the next stage. In this scenario, two `BlockingQueue` instances are used. The first is the `checkedShippingBoxQueue` queue, which contains the shipping box with its contents checked. The program will dequeue the first element in the queue using the `take()` method. If the queue is empty, the thread waits until a new shipping box is available to process. Once the shipping boxes are labelled, the box will be inserted into the labelled `ShippingBoxQueue` queue using the `put()` method. Implementing `BlockingQueue` for such scenarios enables robust yet straightforward communication across threads, allowing threads to perform smoothly without requiring programmers to lock and release the queues manually.

- **Concurrent Hash Map**

Java has also provided the `ConcurrentHashMap` class that allows multiple threads to read and write data concurrently to the hash map without requiring additional external synchronisation, making it preferable to the traditional `HashMap` when concurrency concepts are involved. Before Java 8, `ConcurrentHashMap` used segment-based locking to achieve atomicity, where a `ConcurrentHashMap` will be divided into 16 segments by default, and only the specific segments that are involved in operations will be locked, achieving fine-grained locking in `ConcurrentHashMap` (JMSE, 2015). However, after the update to Java 8, `ConcurrentHashMap` uses a more advanced technique, where the algorithm reduces the reliance on `synchronized` blocks and `ReentrantLock` locks by incorporating compare-and-swap (CAS) techniques, allowing `ConcurrentHashMap` to maintain its atomicity while promoting minimal resource contention (Pramodbablad, 2025). These changes have made `ConcurrentHashMap` thread-safe, so concurrent programs will be able to manage shared records in real-time without leading to race conditions.

```
public class DataManager {  
    public static Map<Item, Integer> inventoryList = new ConcurrentHashMap<>();  
}
```

```
public class Shelf {  
    public static void removeItems(Item item, int quantity) {  
        if (item == null || quantity <= 0) throw new IllegalArgumentException("Invalid item or quantity.");  
  
        DataManager.inventoryList.compute(item, (_, currentStock) -> {  
            if (currentStock == null) throw new IllegalArgumentException("Item does not exist.");  
            if (currentStock < quantity) throw new IllegalArgumentException("Insufficient stock.");  
            return currentStock - quantity;  
        });  
    }  
}
```

The code snippet above demonstrates the use of `ConcurrentHashMap` to record the number of items in an inventory. As multiple picking arms will be retrieving items from the inventory shelf to be placed into order bins, where such a scenario is simulated by having three threads running the `PickingArms` task, it is necessary for the related methods, especially `removeItems()` from the `Shelf` class, to be thread-safe to ensure that any updates are performed atomically. By using the `compute()` method, which is an atomic update method, the system is able to perform checking on the available inventory and update the new quantity of the item accurately without leading to conflicts.

Safety Aspects in Program Execution

This section details the different techniques applied to ensure that the warehouse system simulation remains thread-safe and free from concurrency issues.

Producer-Consumer Model

The producer-consumer model is adapted to ensure different threads will be able to access shared resources in a thread-safe manner. In the producer-consumer model, two types of threads are involved, which are producer threads that generate data or objects, and consumer threads that process the data or objects and complete the tasks. An example from the program is the `TruckGenerator` and `TruckParking` classes, where the `TruckGenerator` class acts as the producer that creates trucks and places them in a `BlockingQueue` called `truckQueue`, while the `TruckParking` class acts as the consumer to retrieve the trucks from `truckQueue` and parks them on loading bays. Such a separation in responsibilities not only simplifies the process of synchronisation, but also enforces the separation of concerns, encouraging a more modular design in concurrency programs (Java Design Patterns, 2025).

```
public class TruckGenerator implements Runnable {  
  
    @Override  
    public void run() {  
        try {  
            while (isRunning) {  
                try {  
                    // Other code here  
                    Truck truck = new Truck();  
                    DataManager.truckQueue.put(truck);  
                    Logger.generateLog(Logger.Type.CREATE_TRUCK, truck);  
                }  
            }  
        } catch (InterruptedException _) {  
            // Code to handle exception  
        } finally {  
            // Code to handle termination  
        }  
    }  
}
```

```
public class TruckParking implements Runnable {  
  
    @Override  
    public void run() {  
        Truck retrievedTruck = null;  
        long startWaitingTime = 0;  
        try {  
            while (!shouldTerminate.get()) {  
                retrievedTruck = DataManager.truckQueue.take();  
  
                // Other code here ...  
            }  
        }  
    }  
}
```


Race Condition Avoidance

Race conditions occur in concurrent programming when two or more threads access a shared resource at the same time, which leads to unpredictable and inconsistent output due to the outcome depending on the sequence or timing of their execution (Lutkevich, 2021). In the simulation of SwiftCart's warehouse system, multiple concurrency constructs like `Semaphore`, `BlockingQueue`, and the `synchronized` keyword have been implemented to prevent multiple threads from concurrently accessing the same variable or critical sections, as shown in the code snippet for the `PickingArms` task below.

```
public class PickingArms implements Runnable {

    private static final Random randomObject = new Random();

    @Override
    public void run() {

        try {
            while (true) {

                // Other code here ...

                // Taking items from queue
                OrderBin binToUse = DataManager.orderBinQueue.take();

                // Acquiring a semaphore
                DataManager.pickingRoboticArmSemaphore.acquire();

                try {

                    // Using a concurrent hash map
                    Map<Item, Integer> orderItemList = new ConcurrentHashMap<>();
                    for (Map.Entry<Item, Integer> orderItem : order.getItemList().entrySet()) {
                        Item item = orderItem.getKey();
                        int quantity = orderItem.getValue();
                        if (randomObject.nextDouble() < WRONG_DETAIL_PROBABILITY) continue;
                        Shelf.removeItems(item, quantity);
                        orderItemList.put(item, quantity);
                        Logger.generateLog(PICKING_FROM_SHELF, item, quantity, order);
                    }

                    // Other code here ...

                } catch (IllegalArgumentException _) {

                    // Code to handle exception

                } finally {

                    // The semaphore is released before proceeding to the next iteration
                    DataManager.pickingRoboticArmSemaphore.release();
                }

            }

        } catch (InterruptedException _) {
            // Code to handle exception
        } finally {
            // Code to handle termination
        }

    }

}
```

Thread Scheduling Fairness

Scheduling fairness is a critical aspect of concurrency programming that ensures that each thread will have an equal opportunity to access shared resources. Ensuring scheduling fairness not only helps avoid concurrency issues like starvation, but also helps to ensure that all threads are fully utilised to improve the system's overall performance (Smith, 2025).

```
public class TruckDispatch implements Runnable {

    public static volatile boolean stopFlag = false;
    private static final AtomicBoolean alternateFlag = new AtomicBoolean(true);

    @Override
    public void run() {
        try {
            while (!stopFlag) {
                boolean currentFlag = alternateFlag.getAndSet(!alternateFlag.get());
                Bay retrievedBay = currentFlag ? DataManager.bayOne : DataManager.bayTwo;

                if (retrievedBay.getCurrentTruck() != null &&
                    retrievedBay.getCurrentTruck().getRemainingCapacity() == 0) {

                    retrievedBay.truckDeparted();
                    Logger.generateLog(
                        Logger.Type.TRUCK_DEPARTED, retrievedBay.getCurrentTruck()
                    );
                    DataManager.trucksDispatchedCounter.incrementAndGet();
                    TruckParking.notifyParkingAvailable();
                }
            }
        } finally {
            // Code to handle termination
        }
    }

    public static void stopRunning() {
        stopFlag = true;
    }
}
```

In SwiftCart's warehouse system context, thread scheduling fairness is considered when designing the operations. An example is the `TruckDispatch` class shown in the above code snippet, where the class is used to check the conditions of the trucks at the two bays in the system to determine if trucks have to be dispatched. A flag called `alternateFlag` tracks the bay that the thread should check. If the `alternateFlag` has the value of `true`, it will proceed to check for the condition of `bayOne`; otherwise, it will check for the condition of `bayTwo`. Once a check is done, the `alternateFlag` switches its value to ensure the other bay is checked during the next iteration. Such is how to implement a simple alternation strategy to employ scheduling fairness in the system that allows `bayOne` and `bayTwo` to be checked without repetitiveness.

Thread Shutdown Protocol

In concurrent systems, it is necessary to gracefully handle thread shutdown to ensure that all threads in the system can complete their tasks without leaving shared resources in an inconsistent state, potentially leading to memory leaks. By having a properly designed shutdown protocol, any unfinished tasks will finish their execution to ensure that all resources are handled and cleaned up properly, improving the system's stability (Goebelbecker, 2024).

```
public class ThreadManager {
    public static void joinAll() throws InterruptedException {
        populateOrderThread.join();
        firstRejectOrderThread.join();
        secondRejectOrderThread.join();
        packingArmThread.join();
        packingScannerThread.join();
        supplyBoxLabellingThread.join();
        boxLabelCheckThread.join();
        collectBoxesToBatches.join();
        containerGenerationThread.join();
        truckGenerationThread.join();
        truckParkingThread.join();
        truckLoadingThread.join();
        truckDispatchThread.join();

        System.out.println("\n" + "~".repeat(120));
        shutdownExecutorService(pickingArms, "Picking Arms");
        shutdownExecutorService(agvArms, "AGV Loaders");
        DataManager.setSystemEndTime();
        System.out.println("All threads have been terminated successfully!");
        System.out.println("=".repeat(80));
    }

    private static void shutdownExecutorService(
        ExecutorService executorService, String serviceName) {

        if (executorService == null) return;
        executorService.shutdown();

        try {
            if (!executorService.awaitTermination(5, TimeUnit.SECONDS)) {
                System.out.println("Forcing shutdown of " + serviceName + "...");
                executorService.shutdownNow();

                if (!executorService.awaitTermination(5, TimeUnit.SECONDS)) {
                    System.err.println(serviceName + " did not terminate cleanly");
                }
            }
            System.out.println(serviceName + " executor service shutdown completed.");
        } catch (InterruptedException _) {
            executorService.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

As shown in the code snippet above, the `join()` method ensures that the `Main` thread will wait for all the user-defined threads to complete their tasks before ending the program's execution. The `shutdown()` method is also called on `ExecutorServices` to ensure that all `ExecutorService` objects can shut down after waiting for a fixed time.

Conclusion

In summary, the simulation of SwiftCart's warehouse system has effectively demonstrated the use of concurrency concepts in Java to ensure the reliability and efficiency of operations in the system. Using numerous constructs like `BlockingQueue`, `synchronized` blocks, and atomic variable classes, the system can handle multiple tasks simultaneously while avoiding concurrency issues like race conditions and starvation with the implementation of various techniques to ensure fair thread scheduling. Such an implementation will assist SwiftCart in identifying the potential bottlenecks that the e-commerce hub might encounter while implementing its warehouse system in a real-world setting, enabling the company to properly design its systems while ensuring all resources are appropriately handled to ensure system stability and consistency.

References

- CodeSignal. (2015). *Introduction to Using Semaphores for Resource Management*. Retrieved from <https://codesignal.com/learn/courses/advanced-concurrency-utilities/lessons/using-semaphores-for-resource-management>
- Coffey, N. (2021). *Explicit locks in Java 5*. Retrieved from Javamex: https://www.javamex.com/tutorials/synchronization_concurrency_9_locks.shtml
- DataCamp. (2025). *Java Threads*. Retrieved from <https://www.datacamp.com/doc/java/threads>
- Drosopoulou, E. (2024, February 7). *Exploring Java's Compare-and-Swap (CAS) for Atomic Operations*. Retrieved from Java Code Geeks: <https://www.javacodegeeks.com/2024/02/exploring-javas-compare-and-swap-cas-for-atomic-operations.html>
- Goebelbecker, E. (2024, September 7). *How to Kill a Java Thread*. Retrieved from Baeldung: <https://www.baeldung.com/java-thread-stop>
- Java Design Patterns. (2025). *Producer-Consumer Pattern in Java: Streamlining Production and Consumption Processes*. Retrieved from <https://java-design-patterns.com/patterns/producer-consumer/>
- Jenkov, J. (2023, June 27). *Java Volatile Keyword*. Retrieved from Jenkov.com: <https://jenkov.com/tutorials/java-concurrency/volatile.html>
- JMSE. (2015, April). *ConcurrentHashMap in java - with Segments formation in detail with diagram*. Retrieved from <https://www.javamadesoeasy.com/2015/04/concurrenthashmap-in-java.html>
- Lahmidi, M. T. (2024, May 6). *Pros and Cons of Implementing Runnable vs. Extending Thread Class in Java*. Retrieved from Medium: <https://medium.com/@mtl98/pros-and-cons-of-implementing-runnable-vs-extending-thread-class-in-java-45705959c802>
- Lutkevich, B. (2021, June 16). *Race condition*. Retrieved from TechTarget: <https://www.techtarget.com/searchstorage/definition/race-condition>

- Microsoft. (2022, November 3). *Threads and threading* . Retrieved from <https://learn.microsoft.com/en-us/dotnet/standard/threading/threads-and-threading>
- Musib, S. (2024, January 27). *LinkedBlockingQueue vs ConcurrentLinkedQueue*. Retrieved from Baeldung: <https://www.baeldung.com/java-queue-linkedblocking-concurrentlinked>
- Pramodbablad. (2025, May 12). *How ConcurrentHashMap Works Internally After Java 8?* Retrieved from Java Concept Of The Day: <https://javaconceptoftheday.com/how-concurrenthashmap-works-internally-after-java-8/>
- Smith, J. A. (2025). *Avoiding Starvation*. Retrieved from <https://publish.obsidian.md/drjerryasmith/Notes/Public/Avoiding+Starvation>
- Tuchin, A. (2024, January 12). *The Challenges and Pitfalls of Using Executors in Java*. Retrieved from DZone: <https://dzone.com/articles/the-challenges-and-pitfalls-of-using-executors-in>

Appendix A. Program Specification

Basic Requirements

1. Order Intake System
 - a. Orders arrive from the online platform at a rate of 1 order every 500ms.
 - b. Each order is verified for payment, inventory availability, and shipping address.
2. Picking Station
 - a. Robotic arms pick items from shelves and place them into order bins.
 - b. Up to 4 orders can be picked at a time.
 - c. Orders are verified for missing items.
3. Packing Station
 - a. Completed bins are packed into shipping boxes (1 order at a time).
 - b. A scanner checks each box to ensure contents match the order.
4. Labelling Station
 - a. Each box is assigned a shipping label with destination and tracking.
 - b. Boxes pass through a quality scanner (1 at a time).
5. Sorting Area
 - a. Boxes are sorted into batches of 6 boxes based on regional zones.
 - b. Batches are loaded into transport containers (30 boxes per container).
6. Loading Bay & Transport
 - a. 3 autonomous loaders (AGVs) transfer containers to 2 outbound loading bays.
 - b. Trucks take up to 18 containers and leave for delivery hubs.
 - c. If both bays are occupied, incoming trucks must wait.

Additional Requirements

1. Defective Orders
 - a. Orders may be rejected at any stage (out-of-stock items, packing errors, mislabeling).
 - b. Defective orders are removed by a reject handler and logged.
2. Capacity Constraints
 - a. The loading bay has space for only 5 containers. If full, packing must pause.
3. Autonomous Loaders
 - a. Only 3 loaders are available and can break down randomly (simulate thread stalls).
 - b. Trucks can only be loaded when a loader and bay are free.
4. Concurrent Activities
 - a. Loaders and outbound trucks operate concurrently.
 - b. Simulate congestion: e.g., 1 truck is waiting while both loading bays are in use.

Appendix B. Output Highlight

Welcome to SwiftCart Warehouse Management System.

Before we start, let's select how you would like to populate the system.

Stress Level

1. Low stress level: 100 orders
2. Normal stress level: 600 orders
3. High stress level: 1000 orders

Please select the level: 2

Now the simulation starts:

```
[Truck-Generation-Thread]
[Order-Generation-Thread]
[Truck-Parking-Thread]
[Picking-Arm-4]
[Picking-Arm-4]
[Packing-Arm-Thread]
[Order-Final-Rejected-Thread]
[Order-Generation-Thread]
[Truck-Parking-Thread]
[Truck-Generation-Thread]
[Picking-Arm-2]
[Picking-Arm-2]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-3]
[Truck-Parking-Thread]
[Truck-Generation-Thread]
[Picking-Arm-3]
[Picking-Arm-3]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-3]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-1]
[Picking-Arm-1]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-1]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-4]
[Picking-Arm-4]
[Picking-Arm-4]
[Picking-Arm-4]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-4]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-2]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-2]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Order-Final-Rejected-Thread]
[Order-Generation-Thread]
[Picking-Arm-3]
[Picking-Arm-3]
[Picking-Arm-3]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-1]
[Picking-Arm-1]
[Picking-Arm-1]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-1]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Order-Reject-First-Time-Thread]
[Order-Reject-First-Time-Thread]
[Picking-Arm-4]
[Picking-Arm-4]
[Picking-Arm-4]
[Picking-Arm-4]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-4]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
[Picking-Arm-2]
[Picking-Arm-2]
[Labelling-Thread]
[Label-Check-Thread]
[Picking-Arm-2]
[Batch-Distribute-Thread]
[Packing-Arm-Thread]
[Packing-Scanner-Thread]
[Order-Generation-Thread]
```

```
] Truck (TRUCK001) is created.
] Order (ORD0001) is created successfully.
] Truck (TRUCK001) is parked at Bay (Bay 1).
] Picking Portable Power Bank with quantity of 4 from the shelf for order (ORD0001).
] Items from order ORD0001 placed into bin BIN001.
] Shipping box of order (ORD0001) is created.
] Order (ORD0001) is rejected with reason: Packing-Scanner-Thread detects different match.
] Order (ORD0002) is created successfully.
] Truck (TRUCK002) is parked at Bay (Bay 2).
] Truck (TRUCK002) is created.
] Picking T-shirt with quantity of 4 from the shelf for order (ORD0002).
] Items from order ORD0002 placed into bin BIN002.
] Shipping box of order (ORD0002) is created.
] Contents from the shipping box of order (ORD0002) is checked.
] Order (ORD0003) is created successfully.
] Picking Notebook with quantity of 1 from the shelf for order (ORD0003).
] No available bay for parking for truck (TRUCK003).
] Truck (TRUCK003) is created.
] Picking Bluetooth Headphones with quantity of 4 from the shelf for order (ORD0003).
] Picking Portable Power Bank with quantity of 2 from the shelf for order (ORD0003).
] Shipping box of order (ORD0002) is labelled.
] Label of shipping box of order (ORD0002) is valid and checked.
] Items from order ORD0003 placed into bin BIN003.
] Shipping box (TRACK0001) disposed to batch CENTRAL_REGION.
] Shipping box of order (ORD0003) is created.
] Contents from the shipping box of order (ORD0003) is checked.
] Order (ORD0004) is created successfully.
] Picking Gaming Keyboard with quantity of 3 from the shelf for order (ORD0004).
] Picking Backpack with quantity of 1 from the shelf for order (ORD0004).
] Shipping box of order (ORD0003) is labelled.
] Label of shipping box of order (ORD0003) is valid and checked.
] Items from order ORD0004 placed into bin BIN004.
] Shipping box (TRACK0002) disposed to batch EAST_MALAYSIA.
] Shipping box of order (ORD0004) is created.
] Contents from the shipping box of order (ORD0004) is checked.
] Order (ORD0005) is created successfully.
] Picking Scented Candle with quantity of 3 from the shelf for order (ORD0005).
] Picking Water Bottle with quantity of 3 from the shelf for order (ORD0005).
] Picking Backpack with quantity of 4 from the shelf for order (ORD0005).
] Picking Phone Charger with quantity of 4 from the shelf for order (ORD0005).
] Shipping box of order (ORD0004) is labelled.
] Label of shipping box of order (ORD0004) is valid and checked.
] Items from order ORD0005 placed into bin BIN001.
] Shipping box (TRACK0003) disposed to batch SOUTHERN_REGION.
] Shipping box of order (ORD0005) is created.
] Contents from the shipping box of order (ORD0005) is checked.
] Order (ORD0006) is created successfully.
] Picking Fitness Tracker with quantity of 2 from the shelf for order (ORD0006).
] Shipping box of order (ORD0005) is labelled.
] Label of shipping box of order (ORD0005) is valid and checked.
] Items from order ORD0006 placed into bin BIN002.
] Shipping box (TRACK0004) disposed to batch NORTHERN_REGION.
] Shipping box of order (ORD0006) is created.
] Order (ORD0006) is rejected with reason: Packing-Scanner-Thread detects different match.
] Order (ORD0007) is created successfully.
] Picking Notebook with quantity of 2 from the shelf for order (ORD0007).
] Picking Yoga Mat with quantity of 3 from the shelf for order (ORD0007).
] Picking Phone Charger with quantity of 2 from the shelf for order (ORD0007).
] Items from order ORD0007 placed into bin BIN003.
] Shipping box of order (ORD0007) is created.
] Contents from the shipping box of order (ORD0007) is checked.
] Order (ORD0008) is created successfully.
] Picking Wireless Mouse with quantity of 1 from the shelf for order (ORD0008).
] Picking Running Shoes with quantity of 1 from the shelf for order (ORD0008).
] Picking Backpack with quantity of 2 from the shelf for order (ORD0008).
] Shipping box of order (ORD0007) is labelled.
] Label of shipping box of order (ORD0007) is valid and checked.
] Items from order ORD0008 placed into bin BIN004.
] Shipping box (TRACK0005) disposed to batch NORTHERN_REGION.
] Shipping box of order (ORD0008) is created.
] Contents from the shipping box of order (ORD0008) is checked.
] Order (ORD0009) is created successfully.
] Order (ORD0009) is rejected with reason: Invalid address. Try again.
] Order (ORD0009) is recreated successfully.
] Picking Gaming Keyboard with quantity of 2 from the shelf for order (ORD0009).
] Picking Notebook with quantity of 2 from the shelf for order (ORD0009).
] Picking T-shirt with quantity of 4 from the shelf for order (ORD0009).
] Picking Water Bottle with quantity of 1 from the shelf for order (ORD0009).
] Shipping box of order (ORD0008) is labelled.
] Label of shipping box of order (ORD0008) is valid and checked.
] Items from order ORD0009 placed into bin BIN001.
] Shipping box (TRACK0006) disposed to batch NORTHERN_REGION.
] Shipping box of order (ORD0009) is created.
] Contents from the shipping box of order (ORD0009) is checked.
] Order (ORD0010) is created successfully.
] Picking Coffee Mug with quantity of 7 from the shelf for order (ORD0010).
] Picking Yoga Mat with quantity of 1 from the shelf for order (ORD0010).
] Shipping box of order (ORD0009) is labelled.
] Label of shipping box of order (ORD0009) is valid and checked.
] Items from order ORD0010 placed into bin BIN002.
] Shipping box (TRACK0007) disposed to batch SOUTHERN_REGION.
] Shipping box of order (ORD0010) is created.
] Contents from the shipping box of order (ORD0010) is checked.
] Order (ORD0011) is created successfully.
```

Truck created and waiting for parking slots at bay.

Orders arrive at rate of 1 order every 500 ms.

4 picking arms place items into order bins.

Orders are verified. Invalid orders are removed by reject handler. Box with content not matching the order is rejected.

Pack order bins into shipping boxes. Shipping boxes are assigned with label. Boxes are sorted into batches based on zones.

